

halfedge: OBJ / PLY / STL / OFF / glTF 读写与网格 构建模块设计文档

src/io.rs

halfedge 库

2026-07-05

目录

1	模块定位	2
2	支持的格式总览	4
3	OBJ 格式约定	4
4	错误类型	4
4.1	错误信息语言约定	5
4.2	数据丢弃警告	5
5	build_mesh_from_vertices_and_faces 算法	5
5.1	整体流程	5
5.2	步骤 1-2: 顶点与内部半边	6
5.3	步骤 3: twin 建立	6
5.4	步骤 4: 边界半边 next/prev 链接 (walking 算法)	6
6	面绕向一致性约束	7
7	OBJ 序列化	8
8	STL 读写	8
8.1	ASCII 格式	8
8.2	二进制格式	9
8.3	ASCII / 二进制自动判别	9
8.4	STL 序列化	9
8.5	StlError	10
8.6	顶点去重原理	10

9	PLY 二进制小端读写	10
9.1	Header 解析	10
9.2	二进制数据布局	11
9.3	ASCII / 二进制自动判别	11
9.4	PlyType 类型分发	11
9.5	format_ply_binary 输出约定	11
9.6	PlyError	12
10	OFF 读写	12
10.1	格式约定	12
10.2	首行解析流程	12
10.3	OffError	13
11	glTF GLB 读写	13
11.1	GLB 文件结构	13
11.2	JSON 与极简 JSON 解析器	14
11.3	GLB 解析流程	14
11.4	Accessor 读取	14
11.5	format_glb 输出约定	15
11.6	GltfError	15
12	统一入口: load_mesh / save_mesh / MeshError	15
12.1	MeshError 枚举	15
12.2	使用示例	16
13	测试覆盖	16
13.1	历史 panic 回归	18

1 模块定位

io.rs 提供**五类网格文件格式**的读写能力:

类别	API	语义
构建	<code>build_mesh_from_vertices_and_faces</code>	顶点 + 三角面索引 → 完整半边网格
	<code>build_mesh_from_polygons</code>	顶点 + 任意多边形面 → 三角化半边网格
OBJ 读	<code>load_obj(path)</code>	从文件加载
	<code>parse_obj(text)</code>	从文本解析
OBJ 写	<code>save_obj(mesh, path)</code>	保存到文件
	<code>format_obj(mesh)</code>	序列化为文本
PLY 读	<code>load_ply(path)</code>	从文件加载（自动判别 ASCII / 二进制）
	<code>parse_ply(text)</code>	从文本解析 PLY ASCII（保留旧入口）
	<code>parse_ply_bytes(bytes)</code>	从字节流解析（自动判别 ASCII / 二进制）
	（内部） <code>parse_ply_binary_le</code>	二进制小端 PLY 解析
PLY 写	<code>save_ply(mesh, path)</code>	保存为 PLY 文件（ASCII）
	<code>format_ply(mesh)</code>	序列化为 PLY ASCII 文本
	<code>save_ply_binary(mesh, path)</code>	保存为 PLY 文件（二进制小端）
	<code>format_ply_binary(mesh)</code>	序列化为 PLY 二进制字节流
STL 读	<code>load_stl(path)</code>	从文件加载（自动判别 ASCII / 二进制）
	<code>parse_stl_bytes(bytes)</code>	从字节切片解析（启发式判别）
	<code>parse_stl_ascii(text)</code>	从文本解析 STL ASCII
	<code>parse_stl_binary(bytes, n)</code>	从字节解析 STL 二进制
STL 写	<code>save_stl_ascii(mesh, path)</code>	保存为 STL ASCII 文件
	<code>format_stl_ascii(mesh)</code>	序列化为 STL ASCII 文本
	<code>save_stl_binary(mesh, path)</code>	保存为 STL 二进制文件
	<code>format_stl_binary(mesh)</code>	序列化为 STL 二进制字节
OFF 读	<code>load_off(path)</code>	从文件加载 OFF（ASCII）
	<code>parse_off(text)</code>	从文本解析 OFF
OFF 写	<code>save_off(mesh, path)</code>	保存到 OFF 文件
	<code>format_off(mesh)</code>	序列化为 OFF 文本
glTF 读	<code>load_glb(path)</code>	从文件加载 GLB（glTF 二进制容器）
	<code>parse_glb(bytes)</code>	从字节流解析 GLB
glTF 写	<code>save_glb(mesh, path)</code>	保存到 GLB 文件
	<code>format_glb(mesh)</code>	序列化为 GLB 字节流
统一入口	<code>MeshError</code>	统一错误枚举（Obj / Ply / Stl / Off / Gltf / Unsuppo
	<code>load_mesh(path)</code>	按扩展名自动选择 OBJ / PLY / STL / OFF / GLB 解析
	<code>save_mesh(mesh, path)</code>	按扩展名自动选择 OBJ / PLY / STL / OFF / GLB 序

2 支持的格式总览

格式	扩展名	变体	特点
OBJ	.obj	ASCII	支持 n-gon、v/vt/vn、负索引
PLY	.ply	ASCII + 二进制小端	自动判别；二进制支持任意属性类型
STL	.stl	ASCII + 二进制	自动判别；顶点按位模式去重
OFF	.off	ASCII	简单文本；支持任意多边形面
glTF	.glb / .gltf	GLB 二进制	最小子集：单 primitive + POSITION + indices

3 OBJ 格式约定

```
1 #          1-based
2 v 0.0 0.0 0.0
3 v 1.0 0.0 0.0
4 v 0.0 1.0 0.0
5 #          / n-gon      v / v/vt / v/vt/vn      v
6 f 1 2 3
7 f 1/1/1 2/2/1 3/3/1
8 f 1 2 3 4 #          n-gon      fan
9 #          OBJ
10 f -3 -2 -1
```

- 仅解析 v 与 f 行；vt、vn、g、usemtl 等其他行被忽略；
- 顶点索引 1-based；负数表示从末尾倒数；
- 支持 n-gon：面顶点数 ≥ 3 即可，自动按 fan 方式三角化；仅当面顶点数 < 3 时返回 `NotTriangular` 错误；
- 索引越界返回 `IndexOutOfRange` 错误。

4 错误类型

ObjError 共 4 个变体：

变体	触发场景
<code>Io(std::io::Error)</code>	文件读写错误
<code>Parse{line, msg}</code>	解析失败（行号 + 描述）
<code>IndexOutOfRange{line, idx, vertex_count}</code>	面索引越界
<code>NotTriangular{line, face_verts}</code>	面顶点数 < 3

4.1 错误信息语言约定

全模块错误信息**统一使用中文**,与代码库其余部分(`panic!`、`eprintln!`、`expect()`、`validate.rs`)保持一致。具体包括:

- 各 Display 实现的标题文字,如 "IO : {e}"、" {line} PLY : {msg}"、"OBJ : {e}";
- Parse{msg} 中的描述串,如 " " : {s}"、" x : {...}"、" {v} {v_count} ";
- Unsupported 中的描述串,如 " PLY"、" PLY : {other}"、" GLB {version} 2 "。

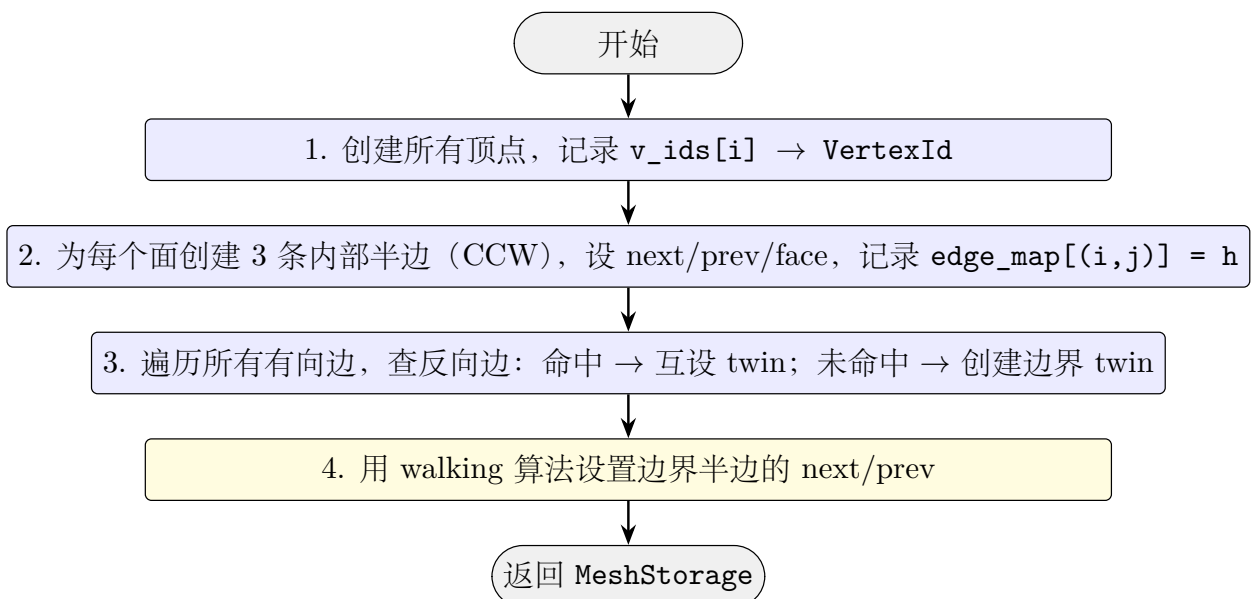
技术术语保留原文: OBJ / PLY / STL / OFF / glTF 格式名、chunk / bufferView / accessor / componentType / POSITION 等 GLTF 规范字段、`h.next` / `twin.prev` 等半边字段路径不在翻译范围内,以保证错误信息与规范文档的可对照性。

4.2 数据丢弃警告

- `build_mesh_from_polygons` 跳过顶点数 < 3 的退化面时,会通过 `eprintln!` 输出警告统计;
- PLY/OFF 解析器不再自行过滤退化面(移除 `if indices.len() >= 3 / if k < 3` 的静默跳过),而是将所有面交给 `build_mesh_from_polygons` 统一处理与警告;
- 所有导出函数(`format_obj` / `format_ply` / `format_ply_binary` / `format_stl_ascii` / `format_stl_binary` / `format_off` / `format_glb`)跳过退化面/非三角面时,会通过 `eprintln!` 输出警告统计。

5 build_mesh_from_vertices_and_faces 算法

5.1 整体流程



容量预分配 入口处调用 `mesh.reserve(vertices.len(), faces.len() * 6, faces.len())` 预分配内存，其中半边容量上限 `faces.len() * 6` 来自每三角面 3 条内部半边 + 3 条潜在边界 twin。`build_mesh_from_polygons` 同样在入口处调用 `reserve`，半边容量按 $\sum_i k_i \times 2$ 计算 (k_i 为第 i 个面的边数)。预分配可将批量构建的 rehash 次数降为 0。

5.2 步骤 1–2：顶点与内部半边

对每个面 $[i_0, i_1, i_2]$ (0-based)，创建三条半边：

$$h_0 : v_{i_0} \rightarrow v_{i_1}, \quad h_1 : v_{i_1} \rightarrow v_{i_2}, \quad h_2 : v_{i_2} \rightarrow v_{i_0}.$$

设置 $\text{next}(h_0) = h_1$, $\text{next}(h_1) = h_2$, $\text{next}(h_2) = h_0$, prev 反向, $\text{face} = f$ 。记录 `edge_map`: $(i_0, i_1) \rightarrow h_0$, $(i_1, i_2) \rightarrow h_1$, $(i_2, i_0) \rightarrow h_2$ 。

面索引越界检查 `build_mesh_from_vertices_and_faces` 与 `build_mesh_from_polygons` 作为公开构建函数，在访问 `v_ids[i]` 之前显式检查 $i < |\text{v_ids}|$ 。若越界，`panic` 并输出索引值与顶点总数（而非让标准库抛出匿名的 “index out of bounds”）。这把 silent panic 转为可诊断的失败，为后续接入 `Result` 变体（缺陷 #17）预留接口。

5.3 步骤 3：twin 建立

对每条有向边 $(a, b) \rightarrow h$ ：

- 查 `edge_map` 是否有 (b, a) ：
 - 命中 h' ：内部边，互设 $h.\text{twin} = h'$, $h'.\text{twin} = h$ ；
 - 未命中：边界边，创建反向边界半边 h^* ($\text{face} = \text{None}$)，互设 twin，加入 `boundary_twins` 列表。

5.4 步骤 4：边界半边 next/prev 链接（walking 算法）

问题：边界半边 bh ($A \rightarrow B$, $\text{face} = \text{None}$) 需要设置 `next` 与 `prev`，使其与其它边界半边构成外环。

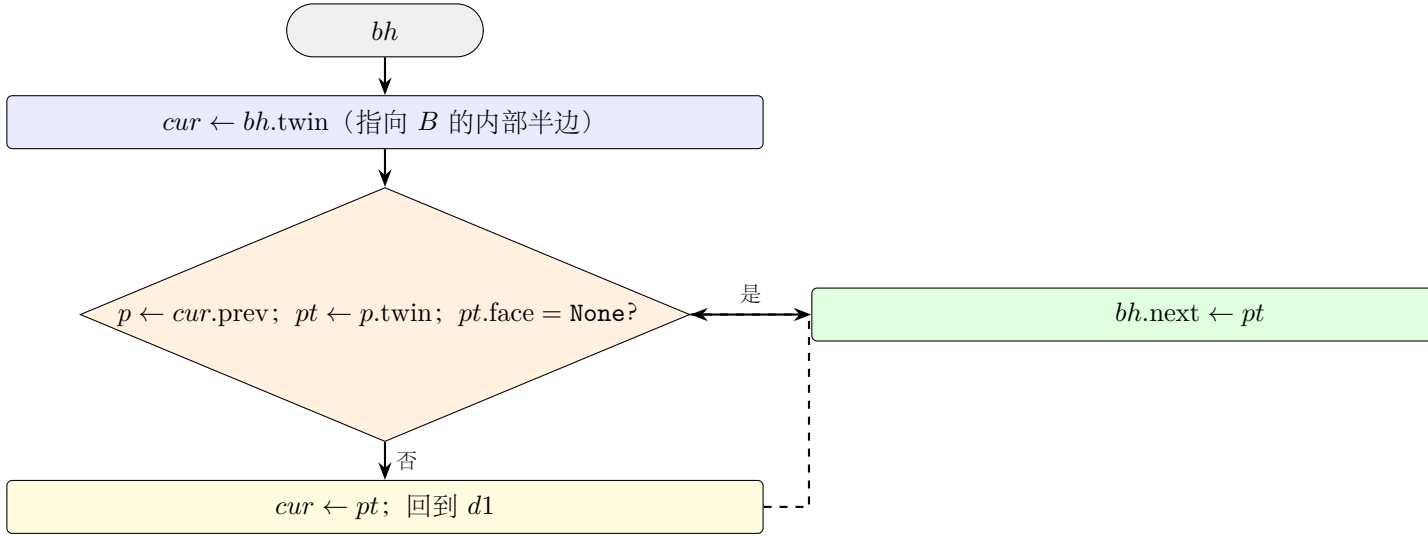
旧公式（仅适用单三角形）：

$$bh.\text{next} = bh.\text{twin}.\text{prev}.\text{twin}, \quad bh.\text{prev} = bh.\text{twin}.\text{next}.\text{twin}.$$

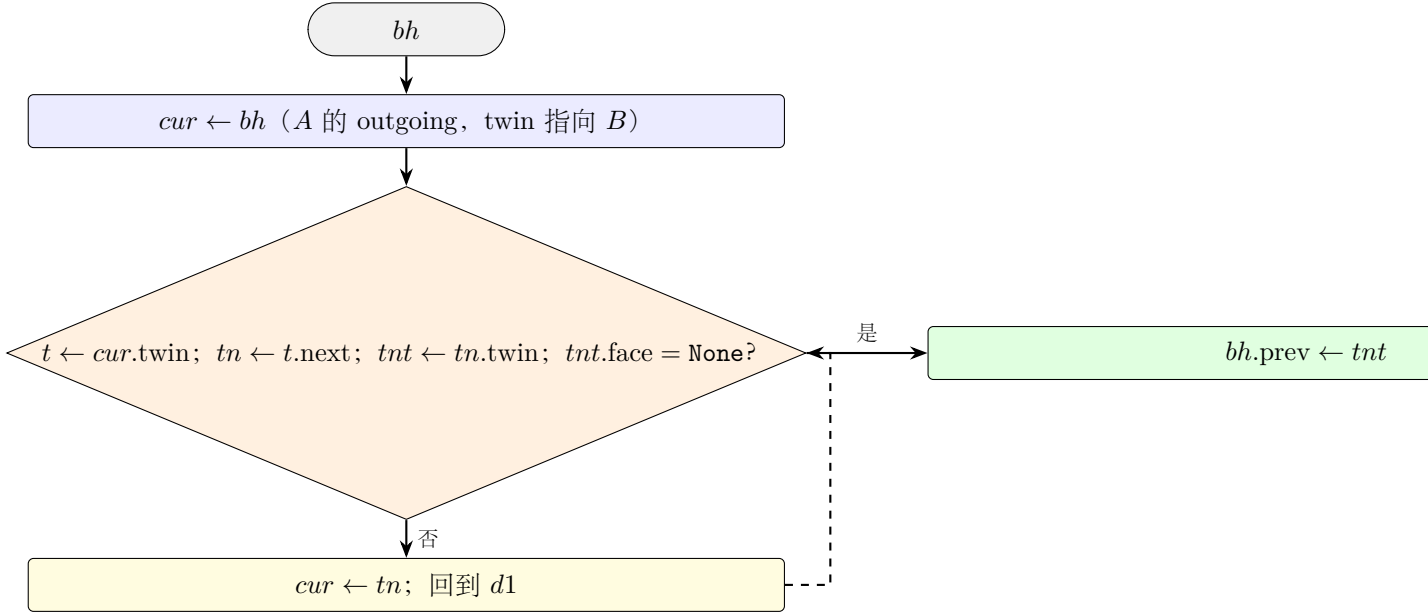
当 $bh.\text{twin}.\text{prev}$ 的 twin 是内部半边时，此公式失效。

新算法（walking）：

求 $bh.\text{next}$ ：绕 $bh.\text{tip}$ （即 B ）沿 CCW 方向旋转，直到找到边界 outgoing 半边。



求 $bh.prev$: 绕 $bh.origin$ (即 A) 沿 CW 方向旋转, 直到找到 $twin$ 为边界半边的 outgoing。



终止性: 边界半边的 tip 必为边界顶点, 边界顶点的 outgoing 环是开链 (非闭合), walking 必终止。代码设 $max_iter = halfedge_count + 1$ 作安全阀。

6 面绕向一致性约束

`build_mesh_from_vertices_and_faces` 假设所有面绕向一致 (每条无向边在两个面中方向相反)。若输入违反此约束 (如两个面都包含有向边 $a \rightarrow b$) , 则:

1. `edge_map` 第二次插入会覆盖第一次;
2. 第一次插入的半边 h_{first} 不在 `directed_edges` 中, 永远不会被设置 `twin`;
3. 第二次插入的半边 h_{second} 查找 (b, a) 失败, 创建边界 `twin`;

4. 结果: $h_{\text{first}}.\text{twin} = \text{None}$, `validate_topology` 会报告 `HalfEdgeDanglingTwin` 或入口不一致。

使用建议:调用方应保证面绕向一致(如所有面 CCW 朝外);若不确定,应先用 `validate_topology` 检查构建结果。

7 OBJ 序列化

`format_obj` 输出格式:

- 顶点按 `vertex_ids()` 顺序, 分配 1-based 索引;
- 面按 `face_ids()` 顺序, 每行 `f i j k`;
- 非三角面 (边界环长度 $\neq 3$) 跳过;
- 顶点位置用 `:.6` 格式化 (6 位小数)。

8 STL 读写

STL (STereoLithography) 格式广泛用于 3D 打印与 CAD 交换。`io.rs` 支持 ASCII 与二进制两种变体, 并提供自动判别。

8.1 ASCII 格式

```
1 solid model
2   facet normal 0.0 0.0 1.0
3     outer loop
4       vertex 0.0 0.0 0.0
5       vertex 1.0 0.0 0.0
6       vertex 0.0 1.0 0.0
7     endloop
8   endfacet
9 endsolid model
```

`parse_stl_ascii` 解析流程:

1. 按行拆分, 跳过 `solid` / `endsolid` / `facet normal` / `outer loop` / `endloop` / `endfacet` 等关键字;
2. 提取所有 `vertex x y z` 行为顶点位置;
3. 每连续 3 个顶点构成一个三角面;
4. **顶点去重:**用 `f64::to_bits()` 的位模式作为 `HashMap` 键, 相同位置的顶点复用同一 `VertexId`;
5. 调用 `build_mesh_from_vertices_and_faces` 构建半边网格。

8.2 二进制格式

```
1 [80          ]
2 [4      little-endian      N]
3 [50      × N      12      + 36      + 2      ]
```

`parse_stl_binary` 解析流程:

1. 跳过 80 字节文件头;
2. 读取 4 字节 `u32` 作为面数 N ;
3. 校验文件长度 $= 84 + 50N$, 不匹配返回 `BadBinarySize` 错误;
4. 每个三角面读取 50 字节, 提取三顶点位置 (法向被忽略, 由网格重建);
5. 顶点去重: 用 `f32::to_bits()` 的位模式 (`u32` 数组) 作为 `HashMap` 键, 相同位置的顶点复用同一 `VertexId`;
6. 调用 `build_mesh_from_vertices_and_faces` 构建半边网格。

8.3 ASCII / 二进制自动判别

`parse_stl_bytes(bytes)` 使用启发式判别:

- 检查文件长度: 若 $\text{len} = 84 + 50N$ (N 为前 80 字节后的 4 字节 `u32`), 判定为二进制;
- 否则尝试以 UTF-8 解码后调用 `parse_stl_ascii`;
- 两者都失败返回 `StlError::Parse`。

为何用文件长度判别? ASCII STL 起始 80 字节通常含 `solid` 关键字, 但二进制文件头也可能是任意 80 字节。文件长度恰好等于 $84 + 50N$ 是二进制的强信号。

8.4 STL 序列化

`format_stl_ascii` 输出标准 ASCII 格式:

- `solid halfedge` 起始;
- 每面输出 `facet normal` (用 `face_normal` 计算) + 三顶点 `vertex`;
- `endsolid halfedge` 结束。

`format_stl_binary` 输出标准二进制格式:

- 80 字节文件头 (前缀 `halfedge binary stl`, 余 0);
- 4 字节 `little-endian` 面数 N ;
- 每面 50 字节: 法向 (12B) + 三顶点 (36B) + 属性 (2B, 全 0)。

8.5 StlError

变体	触发场景
<code>Io(std::io::Error)</code>	文件读写错误
<code>Parse{line, msg}</code>	ASCII 解析失败（行号 + 描述）
<code>BadBinarySize{actual, expected}</code>	二进制文件长度不匹配
<code>NotTriangular{face_verts}</code>	非三角面（顶点数 $\neq 3$ ）

8.6 顶点去重原理

STL 文件每个三角面独立存储三顶点，无共享顶点索引。直接构建网格会产生 $3F$ 个 `VertexId` (F 为面数)，而非实际的 V 个顶点 ($V \approx F/2$)。

去重算法：

$$\text{key}(p) = [\text{f64}::\text{to_bits}(p_x), \text{f64}::\text{to_bits}(p_y), \text{f64}::\text{to_bits}(p_z)]$$

- 用 `HashMap<[u64; 3], u32>` (ASCII) 或 `HashMap<[u32; 3], u32>` (二进制，因 `f32`)；
- `to_bits()` 是位精确比较，浮点数 $0.0 \neq -0.0$ ；
- 命中已有索引则复用，否则分配新 `VertexId`。

为何用位精确而非 ϵ -tolerance?

- STL 文件中相同顶点的浮点表示通常**完全相同**（同一写入器输出）；
- 位精确比较 $O(1)$ 且无歧义；
- 浮点容差比较需 $O(\log V)$ 查找 (KD-tree)，且阈值敏感。

若 STL 文件含微小浮点误差导致同顶点不同表示，应先用 `weld_vertices`（按距离阈值合并）做后处理。

9 PLY 二进制小端读写

PLY 二进制格式在文件大小与解析速度上显著优于 ASCII 变体，是工业应用的首选。本模块支持完整的二进制小端格式 (`format binary_little_endian 1.0`)。

9.1 Header 解析

PLY 文件由**文本 header** 与**二进制数据**两段组成，以 `end_header\n` 为分界。Header 描述了：

- `format: ascii / binary_little_endian / binary_big_endian`；
- `element vertex <N>`：顶点元素数量；
- `property <type> <name>`：顶点属性（按声明顺序排列），支持 `char / uchar / short / ushort / int / uint / float / double` 8 种类型；

- `element face <M>`: 面元素数量;
- `property list <count_type> <index_type> <name>`: 面索引 list。

搜索算法: 用 `windows(11).position(|w| w == b"end_header\n")` 找到字节偏移, 将 header 部分按文本逐行解析后, 剩余字节按二进制布局读取。

9.2 二进制数据布局

顶点数据按 `vertex_stride` 字节连续排列:

$$\text{vertex_stride} = \sum_{i=1}^k \text{size}(\text{type}_i)$$

其中 k 为属性数量, `size(·)` 取 $\{1, 2, 4, 8\}$ 字节。顶点 j 的属性 i 起始偏移:

$$\text{offset}(j, i) = j \cdot \text{vertex_stride} + \sum_{l=1}^{i-1} \text{size}(\text{type}_l)$$

面数据每条由 1 字节 (`uchar`) 顶点数 + $k \cdot \text{size}(\text{index_type})$ 字节索引组成。

9.3 ASCII / 二进制自动判别

`parse_ply_bytes` 通过解析 header 中的 `format` 行决定后续路径:

- `ascii` → 全文件转 UTF-8 后走 `parse_ply_ascii_with_header`;
- `binary_little_endian` → 从 `end_header` 偏移走 `parse_ply_binary_le`;
- `binary_big_endian` → 返回 `Unsupported` 错误。

9.4 PlyType 类型分发

`PlyType` 枚举对 8 种 PLY 标量类型提供:

- `size()`: 字节宽度;
- `read_le_as_f64(bytes)`: 小端读为 f64 (顶点坐标);
- `read_le_as_u32(bytes)`: 小端读为 u32 (面索引);
- `write_le_from_f64(v) / write_le_from_u32(v)`: 小端写字节。

9.5 format_ply_binary 输出约定

输出固定布局 (与 OpenMesh / MeshLab 兼容):

- `format binary_little_endian 1.0`;
- 顶点属性 `float x y z` (每顶点 12 字节);
- 面索引 `property list uchar int vertex_indices` (每面 $1 + 4k$ 字节)。

9.6 PlyError

变体	触发场景
<code>Io(std::io::Error)</code>	文件读写错误
<code>Parse{line, msg}</code>	header 或数据解析失败（行号 + 描述）
<code>Unsupported(String)</code>	不支持的 PLY 特性（如大端）

10 OFF 读写

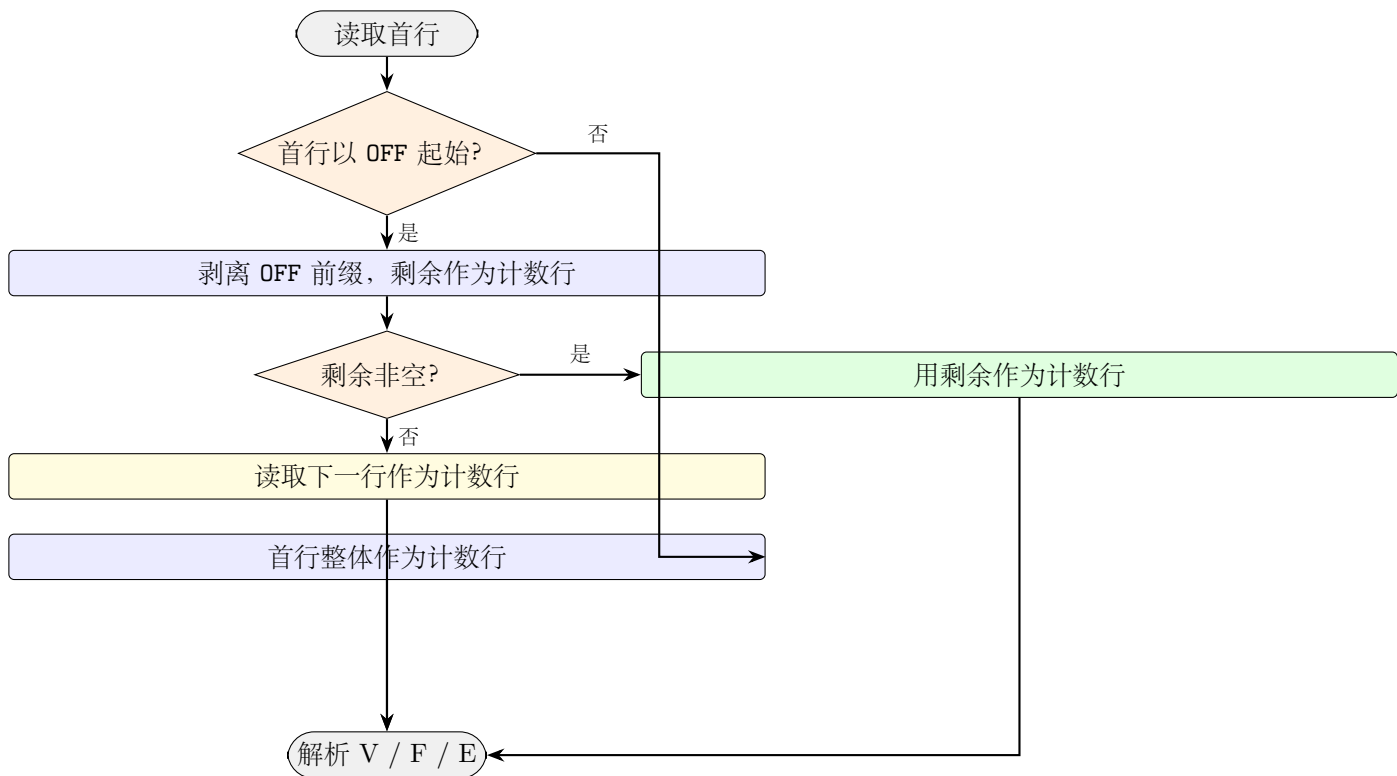
OFF (Object File Format) 是 Geomview 与许多学术工具使用的简化网格格式。

10.1 格式约定

```
1 OFF
2 <vertex_count> <face_count> <edge_count>
3 x y z          # vertex 0
4 ...
5 k v0 v1 ... vk-1    # face 0 (k = vertex count of face)
6 ...
```

- 第一行 **OFF** 关键字可选；部分文件首行为 **OFF 4 4 0**（关键字与计数同行）；
- **#** 开头行视为注释；
- **edge_count** 通常为 0，被忽略；
- 面顶点数 $k \geq 3$ 任意，自动按 fan 方式三角化（通过 `build_mesh_from_polygons`）；
- 索引越界（ $v_i \geq \text{vertex_count}$ ）返回 **Parse** 错误。

10.2 首行解析流程



10.3 OffError

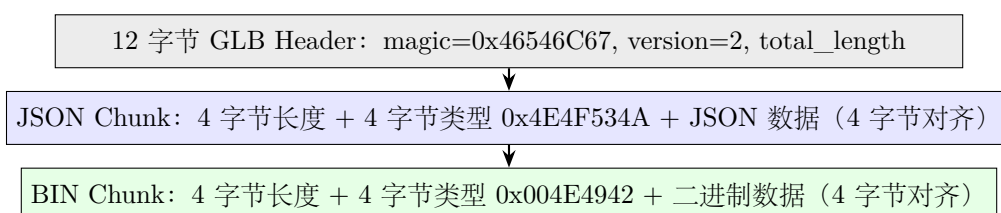
变体	触发场景
<code>Io(std::io::Error)</code>	文件读写错误
<code>Parse{line, msg}</code>	解析失败 (行号 + 描述)

11 glTF GLB 读写

glTF (GL Transmission Format) 是 Khronos 制定的现代 3D 资源交换标准。本模块实现 GLB (glTF 二进制容器) 的**最小子集**:

- 仅支持单 mesh 单 primitive, mode = 4 (TRIANGLES);
- POSITION 属性必须为 FLOAT (componentType 5126);
- 索引 accessor 可选 UBYTE (5121) / USHORT (5123) / UINT (5125), 若无 indices 则按顺序 0..N;
- 不支持材质、纹理、动画、skin、camera、node 层级等。

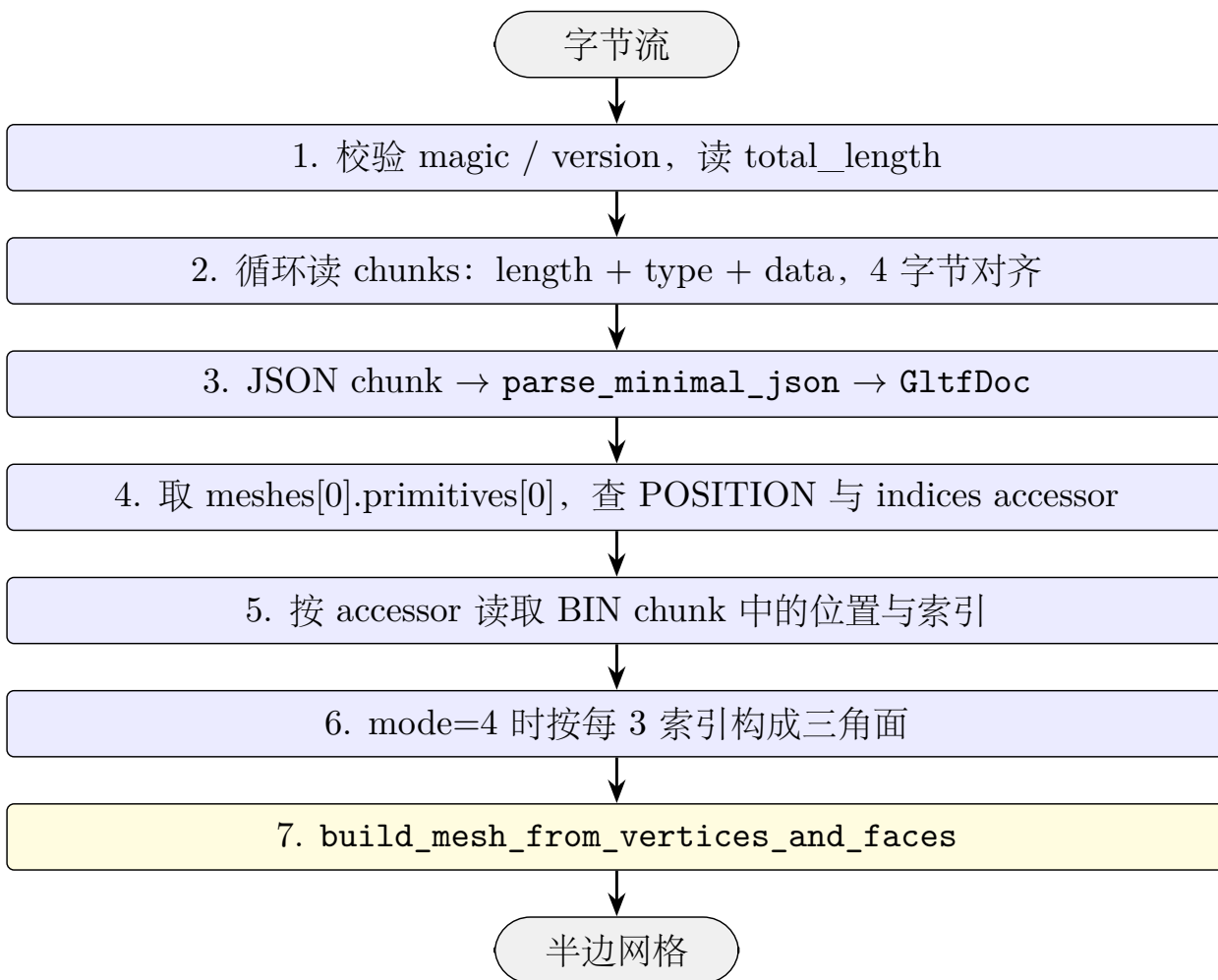
11.1 GLB 文件结构



11.2 JSON 与极简 JSON 解析器

为避免引入 `serde_json` 依赖, 本模块实现一个极简 JSON 解析器 (`parse_minimal_json`), 仅支持 GLB 中所需的对象/数组/原始值。解析树 `JsonValue` 提供 `get(key)` / `as_array()` / `as_u32()` 等访问方法。

11.3 GLB 解析流程



11.4 Accessor 读取

POSITION (`read_accessor_f32x3`):

- 校验 `componentType == 5126 (FLOAT)`;
- 计算起始偏移 $s = \text{bufferView.byteOffset} + \text{accessor.byteOffset}$;
- 每顶点 12 字节 ($3 \times \text{f32 LE}$), 转 `f64` 推入 `Vec<f64; 3>`。

indices (`read_accessor_u32`):

- `componentType ∈ {5121, 5123, 5125}` (`UBYTE` / `USHORT` / `UINT`);
- 按元素大小 1/2/4 字节读为 `u32`。

11.5 format_glb 输出约定

输出固定结构（与 Babylon.js / Three.js 兼容）：

- GLB header: magic=0x46546C67, version=2;
- JSON chunk: 描述 asset / scene / nodes / meshes / buffers / bufferViews / accessors, JSON 文本用空格填充至 4 字节对齐;
- BIN chunk: 先 12V 字节 position ($\text{float32} \times 3V$), 后 4M 字节 indices ($\text{uint32} \times M$), 末尾用 0 填充至 4 字节对齐;
- 仅三角面输出, 非三角面跳过。

11.6 GltfError

变体	触发场景
Io(std::io::Error)	文件读写错误
Parse(String)	GLB 结构或 JSON 解析失败
Unsupported(String)	不支持的 glTF 特性（如 mode $\neq$ 4）

12 统一入口: load_mesh / save_mesh / MeshError

为简化上层调用, io.rs 提供按扩展名自动分派的统一接口：

扩展名	load_mesh 调用	save_mesh 调用
.obj	load_obj	save_obj
.ply	load_ply（自动判别 ASCII / 二进制）	save_ply（ASCII）
.stl	load_stl（自动判别 ASCII / 二进制）	save_stl_ascii
.off	load_off	save_off
.glb / .gltf	load_glb	save_glb
其他	Err(MeshError::UnsupportedFormat(ext))	

12.1 MeshError 枚举

MeshError 是 OBJ / PLY / STL / OFF / glTF 错误的**统一包装**, 通过 From<ObjError> / From<PlyError> / From<StlError> / From<OffError> / From<GltfError> 实现自动转换, 便于在 ? 运算符中混用:

变体	来源
Obj(ObjError)	load_obj / save_obj 错误
Ply(PlyError)	load_ply / save_ply* 错误
Stl(StlError)	load_stl / save_stl_* 错误
Off(OffError)	load_off / save_off 错误
Gltf(GltfError)	load_glb / save_glb 错误
UnsupportedFormat(String)	未知扩展名

12.2 使用示例

```

1 //
2 let mesh = halfedge::load_mesh("model.obj");
3 let mesh = halfedge::load_mesh("model.ply"); // ASCII /
4 let mesh = halfedge::load_mesh("model.off");
5 let mesh = halfedge::load_mesh("model.glb");
6
7 //
8 halfedge::save_mesh(&mesh, "out.ply"); // ASCII
9 halfedge::save_mesh(&mesh, "out.glb"); //
10 halfedge::save_ply_binary(&mesh, "out_bin.ply"); // PLY

```

13 测试覆盖

测试名	验证点
构建器 (10 个)	
build_mesh_basic_quad	4 顶点 2 面 10 半边
build_mesh_passes_full_validation	构建结果通过完整校验
build_mesh_closed_tetrahedron	闭合四面体 12 半边
build_mesh_empty_inputs_returns_empty_mesh	空顶点 + 空面 → 空网格, 不 panic
build_mesh_vertices_no_faces	仅顶点无面 → 顶点保留无面
build_polygons_empty_inputs_returns_empty_mesh	build_mesh_from_polygons 空输入
build_mesh_face_index_out_of_range_panics	面索引越界 panic!(" ")
build_polygons_face_index_out_of_range_panics	多边形入口同样越界 panic
build_polygons_skips_degenerate_face_2_verts	2 顶点退化面被静默跳过
build_polygons_mixed_degenerate_and_valid	退化与有效面混合时只保留有效面
OBJ 往返与解析 (12 个)	
obj_roundtrip_quad	四边形 OBJ 往返一致
obj_roundtrip_tetrahedron	四面体 OBJ 往返一致
obj_parse_skips_comments_and_other_lines	忽略 # / vt / vn / g 等

测试名	验证点
obj_parse_supports_v_vt_vn_format	解析 v/vt/vn 格式
obj_parse_negative_indices	负索引支持
obj_parse_quadrilateral_face_succeeds	四边形面解析成功（支持 n-gon）
obj_parse_out_of_range_index_fails	越界索引报 <code>IndexOutOfRangeException</code>
obj_save_load_file_roundtrip	文件 IO 往返一致
obj_parse_empty_text	空文本 → 空网格，不报错
obj_parse_only_vertices_no_faces	仅 v 行 → 顶点保留无面
obj_parse_face_zero_index_fails	0 索引（OBJ 1-based）报错
obj_parse_face_index_equal_to_vertex_count_fails	索引 = 顶点数报越界
<i>PLY 二进制（6 个）</i>	
ply_binary_roundtrip	二进制 PLY 字节往返一致
ply_binary_file_roundtrip	二进制 PLY 文件 IO 往返
ply_ascii_still_works_via_parse_ply_bytes	旧 ASCII 文本经字节入口仍可解析
ply_binary_detects_bad_header	缺失 <code>end_header</code> 报 <code>Parse</code>
ply_parse_empty_bytes_fails	空字节流报错
ply_parse_missing_end_header_fails	缺失 <code>end_header</code> 报 <code>Parse</code>
<i>STL 往返与解析（9 个）</i>	
stl_ascii_roundtrip	STL ASCII 往返一致
stl_binary_roundtrip	STL 二进制往返一致
stl_ascii_parses_minimum_solid	最小 <code>solid</code> 块解析成功
stl_ascii_rejects_non_triangular	非三角面报 <code>NotTriangular</code>
stl_binary_detects_size_mismatch	二进制长度不匹配报 <code>BadBinarySize</code>
stl_file_roundtrip_ascii	STL ASCII 文件 IO 往返
stl_file_roundtrip_binary	STL 二进制文件 IO 往返
stl_ascii_empty_text	空文本 → 空网格，不报错
stl_ascii_only_solid_endsolid	仅 <code>solid/endsolid</code> 块 → 空网格
<i>OFF 往返与解析（7 个）</i>	
off_roundtrip_tetrahedron	四面体 OFF 往返一致
off_file_roundtrip	OFF 文件 IO 往返
off_parse_counts_inline_with_keyword	首行 OFF 4 4 0 行内计数支持
off_parse_quadrilateral_face_succeeds	四边形面解析成功
off_parse_out_of_range_index_fails	索引越界报 <code>Parse</code>
off_parse_empty_text_fails	空文本报错（OFF 需首行关键字）
off_parse_only_vertices_no_faces	仅顶点无面 → 顶点保留无面
<i>glTF GLB 往返与解析（4 个）</i>	
glb_roundtrip_tetrahedron	四面体 GLB 字节往返一致
glb_file_roundtrip	GLB 文件 IO 往返
glb_detects_bad_magic	错误 magic 报 <code>Parse</code>

测试名	验证点
glb_icosphere_roundtrip	icosphere 经 GLB 往返一致
统一入口 (6 个)	
auto_detect_obj	load_mesh(".obj") 走 OBJ 路径
auto_detect_ply	load_mesh(".ply") 走 PLY 路径
auto_detect_stl	load_mesh(".stl") 走 STL 路径
auto_detect_off	load_mesh(".off") 走 OFF 路径
auto_detect_glb	load_mesh(".glb") 走 GLB 路径
auto_detect_unsupported	未知扩展名返回 UnsupportedFormat
损坏输入与退化面 (11 个, 散布于上述分组)	
上述 *_empty_* / *_out_of_range_* / *_degenerate_* / *_only_* 系列	
覆盖空输入、越界索引、退化面、仅顶点等边界场景,	
确保所有解析入口在异常输入下既不 unwrap panic、也不静默吞错。	

13.1 历史 panic 回归

tests/panic_safety.rs 集成测试中针对 io 模块的回归用例:

- regression_build_mesh_index_out_of_range_panics_with_clear_message: 保证越界 panic! 消息含中文 ;
- regression_parse_obj_empty_input_no_panic: 空 OBJ 文本不 panic;
- regression_parse_obj_only_vertices_no_faces_no_panic: 仅顶点 OBJ 不 panic;
- regression_parse_obj_malformed_face_skipped_no_panic: 损坏面行不 panic。

全模块共 54 个单元测试,全库共 578 个单元测试 + 24 个集成测试(tests/mesh_workflow.rs 与 tests/panic_safety.rs)。